Ders 2 Genişletilmiş Notu: NetBeans Kurulumu, İlk Java Projesi, Paketler ve Referans Mantığı

Ders 2 Genişletilmiş Notu: NetBeans Kurulumu, İlk Java Projesi, Paketler ve Referans Mantığı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Geliştirme Ortamının Kurulumu
- 2. NetBeans'te İlk Java Projesi
- 3. Main Metodu ve Sürücü Sınıf
- 4. İlk Çıktı: println ve print
- 5. System.out.println Aslında Nedir?
- 6. Paketler
- 7. İlk Sınıflar: Kitap ve Yazar
- 8. Bellekte Nesne: Stack ve Heap
- 9. Nesneyi Yazdırmak
- 10. Nesne İçinde Nesne ve Nokta Operatörü
- 11. Referans Mekanizması: Atama, Çöp Toplayıcı ve Kopyalama
- 12. Yapısal Programlamanın Java'daki Yeri
- 13. Debug: Programı Adım Adım Çalıştırmak
- 14. JFrame ve UML'in Ne Olduğu
- 15. IntelliJ'de Swing UI Designer ve PlantUML Denemesi
- 16. Easy UML ile Sınıf Diyagramı

1. Geliştirme Ortamının Kurulumu

Bu oturum uygulamalı bir oturumdu. Hoca önce herkesin aynı geliştirme ortamına sahip olmasını istedi, bu yüzden kurulum adımları derste birlikte yapıldı. Ardından geçen dönemin yapısal programlama konularından yola çıkarak birkaç kod yazıldı.

1.1 NetBeans'i İndirmek

Arama motoruna "NetBeans download" yazılarak indirme sayfasına gidilir. Hoca, projenin artık **Apache NetBeans** adını aldığını ve bu yüzden farklı bir sayfaya yönlendirildiğimizi söyledi. İndirme sayfasında Windows, Linux ve Mac seçenekleri vardır, işletim sistemine uygun olan seçilir. Dosya birkaç yüz megabayt büyüklüğündedir: Hoca önce 300 küsur megabayt beklediğini söyledi, ekranda 400 megabayta yakın bir değer gördü.

1.2 UML Eklentisi Arayışı

İndirme sürerken hoca **Easy UML** adlı eklentiyi aradı. Bu eklentinin resmî desteği NetBeans'in eski bir sürümünde (8.2) kalmıştı. En son güncellemesi 2017'ye aitti, yorumlarda ise birisi 11. sürümde sorunsuz çalıştığını yazmıştı. Yani yeni sürümde çalışıp çalışmayacağı denenmeden bilinmiyordu.

Hoca bu aracı neden istediğini şöyle açıkladı: Araç hem **tersine mühendislik (reverse engineering)** yapıyor, yani bir proje verildiğinde onun UML şemasını çıkarıyor, hem de önce şemayı çizip ondan **kod üretmeye (code generation)** imkân veriyor. Bunlar daha çok yazılım mühendisliği tarafında anlatılan konulardır, ama bu derste de konuşulacaktır. Alternatif olarak yalnızca UML üreten başka yazılımlar da vardır. Eklentinin kurulumu dersin sonunda yapıldı (Bölüm 16).

1.3 Oracle'ın Eğitim Sayfaları

Bekleme sırasında hoca Oracle'ın eğitim sayfalarını (Oracle University / Oracle Academy) tanıttı. Burada **Java Explorer** adında yaklaşık 7 saatlik bir içerik vardır, ayrıca MySQL ve Oracle gibi veri tabanı konularıyla ilgili içerikler de bulunur. İçerikleri izlemek için üyelik gerekir. Hoca üyelik için `.edu` uzantılı e-posta adresi önerdi, çünkü bu tür yerler bazen özel e-posta adreslerini kabul etmeyebiliyor. Tamamlayınca sertifika da verilebileceğini, ama bundan emin olmadığını söyledi. Benzer eğitim sayfaları, C# ve .NET teknolojileri üzerine çalışacaklar için Visual Studio tarafında da vardır.

1.4 NetBeans Kurulum Adımları

Kurulum dosyasına çift tıklandığında bir kurulum sihirbazı açılır. Kurulum basittir, büyük ölçüde ileri demekten ibarettir:

1. **Customize** düğmesiyle hangi bileşenlerin kurulacağı seçilebilir. Hoca örnek olarak Enterprise Edition'ı ve PHP'yi çıkardı, HTML ve JavaScript'i bazen kullandığı için bıraktı. İsteyen hepsini seçebilir.
2. **Next** ile devam edilir ve lisans kabul edilir.
3. JDK ekranı gelir (Bölüm 1.5).
4. **Next** denir, güncellemelerin yüklenmesi seçilir ve **Install** ile kurulum başlatılır.

1.5 JDK Gerekliliği

NetBeans'in çalışması için **JDK (Java Development Kit)** gerekir. Hocanın ekranında JDK zaten bulunduğu için kurulum onu doğrudan gördü. Hoca bunun sebebini sonradan anladı: Ders için kendi NetBeans'ini kaldırırken JDK kaldırılmamıştı.

Soru: JDK'yı ayrıca indirmek gerekiyor mu? Kurulum dosyasına çift tıklayınca "JDK'nız yok" diyor ve Oracle'ın sitesine yönlendiriyor. **Cevap:** Evet. JDK'sı olmayanlar yönlendirilen yerden indirip kurmalı, sonra NetBeans kurulumuna devam etmeli.

Hangi sürüm sorusuna hoca "olabildiği kadar en son sürüm olsun" dedi. Bununla birlikte derste anlatılacak konuların son sürümlerin çok gerisinde kaldığını da ekledi. Derste JDK 16 sayfası açılmayınca ("Page not found") indirilebilen en son sürüm olan **JDK 15** seçildi ve Windows için `.exe` dosyası indirildi. Hoca, slaytta anlatılan kodun derlenmesi işlemlerinin bu araçlarla yapıldığını hatırlattı.

2. NetBeans'te İlk Java Projesi

NetBeans açıldığında menü yapısının Dev-C'ninkine benzediğini gördük. Yeni proje şu adımlarla oluşturulur:

1. **File** menüsünden **New Project** seçilir.
2. **Java with Ant** altından **Java Application** seçilir (Class Library değil), **Next** denir.
3. **Create Main Class** işaretli bırakılır, gerekirse sınıfa ad verilir.
4. **Finish** ile proje oluşturulur.

2.1 Ant, Maven ve Gradle

Yeni proje ekranında Java with Maven, Gradle ve Ant seçenekleri vardır. Hoca bunu şöyle açıkladı: Bir proje yalnızca kaynak kodlardan oluşmaz, projenin derlenmesi, debug edilmesi ve çalıştırılmasıyla ilgili yapılandırma dosyaları da içerir. Bu dosyaları düzenleyen alt yazılımlar, yani kütüphaneler vardır. Ant bunlardan biridir. Maven ve Gradle biraz daha kapsamlı ve

ayrıntılı dosyalar kullanır. Bu ders için şimdilik Ant yeterlidir. Ant'ın altında görsel programlamayla ilgili JavaFX uygulamaları da bulunur, onlar ileride görülecektir.

2.2 Main Class ve Sınıf Adının Yazımı

Sihirbazın sorduğu **Main Class**, içinde `main` metodu olan sınıftır, yani programın açılacağı ilk sınıftır. Sihirbaz bu sınıfa ad vermemize de izin verir. Yazılan adda **son noktaya kadarki kısım bulunduğumuz klasörün (paketin) adı, sonraki kısım sınıfın adıdır**. Derste proje adı `JavaApplication3` olduğu için tam ad `javaapplication3.Deneme` biçiminde kullanıldı.

2.3 Dosya Adı, Sınıf Adı ve Refactor

Hoca derste yanlışlıkla Enter'a basınca sihirbaz main sınıfını proje adıyla aynı adda otomatik üretti. Bu, ad değiştirmeyi göstermek için iyi bir fırsat oldu. Sınıfın adını değiştirmek için dosyaya sağ tıklanır, **Refactor** ve ardından **Rename** seçilir. Refactor "yeniden yapılandırmak" demektir. Bu yolla ad değiştirildiğinde ad kodun içinde de her yerde değişir.

Hoca sonra elle `class deneme2` yazdığında ortamın hata verdiğini gösterdi: Sınıfın adı dosyanın adıyla aynı olmalıdır. Sebep olarak kodun bytecode haline dönüşeceğini ve bu aşamada sorun çıkmaması için adların aynı olması gerektiğini söyledi. Dosyanın başındaki yorum satırları ise otomatik gelir.

> **Ders sonrası not:** Hoca sınıf adı ile dosya adının aynı olması gerektiğini genel bir kural olarak söyledi; doğrusu bu zorunluluk `public` sınıflar içindir. javac ile denendi: `Deneme.java` içinde `public class Deneme2` yazınca "class Deneme2 is public, should be declared in a file named Deneme2.java" hatası alındı. Aynı dosyada `public class Deneme` yanında `public` olmayan `class Yardimci` ise hatasız derlendi ve iki ayrı `.class` dosyası üretti.
>
> **Ek bilgi (derste anlatılmadı):** `.java` uzantılı dosya kaynak koddur, `javac` onu bytecode içeren `.class` dosyasına çevirir. javac ile denendi: `Deneme.java`, `Kitap.java` ve `Yazar.java` derlenince her sınıf için ayrı bir `.class` dosyası oluştu. Bu dosyalar kaynak kod olmadığı için üzerinde çalışılacak olan `.java` dosyalarıdır.

2.4 Proje Klasörü

Projenin diskte nereye oluştuğunu görmek için proje özelliklerine (**Properties**) bakılır. Windows'ta konum, kullanıcı klasörünün altındaki Belgeler (Documents) içindeki `NetBeansProjects` klasörüdür. Bunun altında proje adıyla bir klasör vardır. Klasör yapısı şöyledir:

```
Belgeler/
  NetBeansProjects/
    JavaApplication3/
      (projeyle ilgili yapılandırma dosyaları)
      src/
        javaapplication3/
          Deneme.java
```

Kodumuz `src` altındadır. Orada paket adıyla bir klasör ve onun içinde `.java` dosyalarımız bulunur. Birazdan eklenecek sınıflarla burada birden fazla `.java` dosyası olacaktır.

Soru: İlk açılışta proje olarak ne seçiyorduk? **Cevap:** File, New Project, Java Application.

Soru: Bende Java dosyası oluşmadı, Source Packages altında sizdeki gibi bir paket göremiyorum. **Cevap:** File, New Project, Java with Ant ve Application seçilip Next denir, Create Main Class ile Finish'e basılır. Sonra soldaki menüde proje görünmelidir. Soruyu soran öğrencide adımlar tekrarlanınca dosya oluştu.

2.5 NetBeans'te C ve C++

Bilgisayarda bir C derleyicisi varsa NetBeans'te C/C++ kodu da yazılabilir. Örneğin Dev-C'nin derleyicisi gösterilerek NetBeans bir editör gibi kullanılabilir.

3. Main Metodu ve Sürücü Sınıf

C'de `int main` vardı, burada da bir `main` var. `main` metodunun bulunduğu sınıf, çalıştırılacak ilk dosyadır. Hoca bunun genel bir kural olduğunu vurguladı: Görsel programlamada da, Java ile yazılan web programlarında da program `main` olan sınıftan başlar. Bir uygulamada bir tane `main` olmasını bekleriz. İkinci ya da üçüncü bir `main` eklenirse NetBeans hangisinden başlatmak istediğimizi sorar.

Bu özel sınıfa **sürücü sınıf (driver class)**, **test edici (tester)** ya da **test sınıfı (test class)** denir. Görevi, projedeki diğer sınıfları test etmek, yani onları "sürmek"tir. Projeye başka sınıflar da eklenecektir, ama onlarda `main` metodu olmayacaktır. Onlar bu `main` üzerinden kullanılacaktır.

> **Ders sonrası not:** Hoca Java ile yazılan web programlarının da `main` olan sınıftan başladığını söyledi. Büyük olasılıkla servlet tabanlı web uygulamalarında geliştiricinin yazdığı bir `main` bulunmaz, başlangıcı uygulamayı çalıştıran sunucu yönetir. Hocanın söylediği kural, bu derste yazacağımız masaüstü uygulamaları için geçerlidir.

4. İlk Çıktı: println ve print

İlk program olarak ekrana bir şey yazdırıldı. Derste yazılan sürücü sınıf şöyleydi. Sınıfın sonraki satırları ileriki bölümlerde anlatılıyor, burada dersin sonundaki hali tek parça verilmiştir:

```
package javaapplication3;

public class Deneme {

    public static void main(String[] args) {
        System.out.println("deneme");
        System.out.print("denemeee");   // alt satıra geçmez
        System.out.println("sdfsdf");

        int x = 5;
        System.out.println(x);          // değerini yazar

        Yazar y = new Yazar();
        System.out.println(y);          // paket.Sinif@... biçiminde yazar

        Kitap k = new Kitap();
        System.out.println(k);
    }
}
```

Programı çalıştırmanın birkaç yolu vardır: Dosyaya sağ tıklayıp **Run File** demek, araç çubuğundaki **Run** düğmeleri ya da klavye kısayolu. Hocanın deyişiyle beş tane yol vardır, hangisi istenirse o kullanılır. Çalıştırınca alt taraftaki pencerede programın çıktısı ve sonunda ne kadar sürdüğüyle birlikte **Build Successful** yazısı görünür, bu programın tamamen bittiğini gösterir.

`println` ile `print` arasındaki fark tahmin edileceği gibidir: `println` yazdıktan sonra alt satıra geçer, `print` geçmez. İkisinin başka önemli bir farkı yoktur. Yukarıdaki kodun çıktısı `javac` ile derleyip çalıştırılarak denendi:

```
deneme
denemeeesdfsdf
5
javaapplication3.Yazar@1dbd16a6
javaapplication3.Kitap@251a69d7
```

`print` alt satıra geçmediği için `denemeee` ile `sdfsdf` bitişik çıkar. Son iki satırdaki `@` işaretinden sonraki değerler her çalıştırmada değişebilir.

4.1 sout Kısayolu

`System.out.println` yazmak uzun olduğu için ortamın hızlandırıcıları vardır. NetBeans'te `sout` yazıp **Tab** tuşuna basınca `System.out.println("");` kendiliğinden oluşur, sonra içine yazılacak şey yazılır. Hoca derste yazarken kısalık için `s.o.p` yazdığını söyledi. Başka kısayollar için "NetBeans shortcuts" diye aranabilir. Dokümantasyona göre Eclipse'te `syso` ve Ctrl+Space, JDeveloper'da `sop` ve Ctrl+Enter aynı işi yapar.

Soru: `sout`'tan Tab yapınca `System.out` geldi. Bunun için bir kütüphane çekmemiz mi lazım? **Cevap:** Hayır, bu NetBeans'in kendi özelliğidir, bir kütüphaneyle ilgisi yoktur.

Soru: Bu kısayollardan kendimiz oluşturabilir miyiz? **Cevap:** Hoca şimdiye kadar denemediğini, kısayol tanımlama imkânı olduğunu tahmin ettiğini, araçların menülerinde (Tools gibi) bakılması gerektiğini söyledi. Kesin cevap derste verilmedi.

Giriş ve çıkış konusunun ayrıntısına hoca şimdilik girmedi, yalnızca çıktı tarafına baktı.

5. System.out.println Aslında Nedir?

C'de `printf` görmüştük. Burada ise arada noktalar olan bir yazım var: `System`, sonra yeşil yazılan `out`, sonra `println`. Hoca bu yazımın ne anlama geldiğini NetBeans'te tanımına giderek gösterdi. Bir ifadenin üzerindeyken **Navigate, Go to Source** seçilirse (ya da Ctrl tuşuyla tıklanırsa) tanımın bulunduğu yere gidilir.

Bu yolla `System` sınıfına gelindi. Orada `PrintStream  out` diye bir tanım görüldü, `out` da `PrintStream`'e götürdü. Ayrıntıların bir kısmı kapalıydı, ekranda **compiled code** ifadesi görülüyordu. Bu, bakılan kodun Java derleyicisi tarafından derlenmiş halde olduğunu gösterir. Hoca, bunların kaynağını açık olarak gösteren siteler de olduğunu söyledi.

Sonuçta yapı şöyledir: `System` bir sınıftır. `out`, bu sınıfın altındaki statik bir değişkendir ve bir nesnedir. `println` ise bu nesnenin bir metodudur. Hoca, `System`'in `out`'u aracılığıyla işletim sistemiyle haberleşme yapıldığını anlattı: Metoda gelen argüman işletim sistemi vasıtasıyla çıkış akışına (output stream) gönderilir. Giriş söz konusu olduğunda da giriş akışından (input stream) alınır.

> **Ders sonrası not:** Hoca önce `out` için "alt sınıf, inner class" ve "statik bir sınıf" dedi, sonra koda bakıp kendisi "class değilmiş, bir nesne" diye düzeltti. Doğrusu bu düzeltmedir. Java ile denendi (yansıma ile alan bilgisi okundu): `public static final java.io.PrintStream java.lang.System.out`.

5.1 Method Overloading'e İlk Bakış

`PrintStream` içinde hoca `println` ve `print` metotlarının birçok kez tanımlandığını gösterdi: `long` için, `int` için ayrı ayrı. Metotların imzası bir yere kadar aynıdır, parametre kısmında değişir. Aynı adla farklı parametrelere sahip metotlar tanımlanmasına **metot aşırı yükleme (method overloading)** denir. Bu konu ileride görülecektir.

> **Ek bilgi (derste anlatılmadı):** Java ile denendi (yansıma ile metot listesi okundu): JDK 25'te `PrintStream` sınıfında 10 tane `println` vardır. Parametreleri: parametresiz, `boolean`, `char`, `int`, `long`, `float`, `double`, `char[]`, `String` ve `Object`. Bu yüzden `println(x)` ile `println(y)` aynı adı kullanmalarına rağmen farklı sürümlere gider.

6. Paketler

Paket (package), projeyi fiziksel olarak değil **mantıksal olarak** parçalara bölmeyi sağlar. Hocanın örneği bir üniversite sistemi projesiydi. Bu projede muhasebeyle ilgili kısımlar, öğrencilerle ilgilenen birinin alanı değildir, onlar ayrı bir yerde tutulur. Öğrencilerin notlarıyla ilgili kısımlar da ayrı bir pakette durur.

Paketler arasında aynı adlı sınıflar olabilir. Örneğin hem muhasebe paketinde hem de öğrenci paketinde `Yazdir` diye bir sınıf bulunabilir. Bunlar farklı şeylerdir ve paketler sayesinde mantıksal olarak birbirinden ayırt edilir. Dosyanın tam adı da paketle birlikte söylenir: Bu dosya aslında `javaapplication3.Deneme` dosyasıdır.

Başka bir paketteki dosyaya erişmek için o paketin bulunduğumuz dosyaya dahil edilmesi gerekir. Hoca bunun ayrıntısına daha sonra geleceğini söyledi.

7. İlk Sınıflar: Kitap ve Yazar

7.1 Projeye Sınıf Eklemek

Paket üzerinde sağ tıklanıp **New, Java Class** seçilir. Boş bir Java sınıfı istiyoruz. Sınıfa ad verilir ve **Finish** denir. Bu ekranda üst sınıf (superclass) ya da arayüz (interface) belirtme seçenekleri de vardır, ama hoca öğrenmek adına bunları arayüzden değil, elle yazacağımızı söyledi. Oluşan dosyada NetBeans `public class` tanımını yazıp bırakır. `Deneme` de aynı biçimdedir, tek farkı içinde `main` metodu olmasıdır.

Sınıflarla nesnelerin özelliklerini, C'deki yapılarda (`struct`) olduğu gibi bir arada yazacağız. Sınıfların `struct`'tan farkı, nesnelere ek özellikler de katacak olmamızdır.

7.2 Yazar String Olmalı mı?

Hoca kitap sınıfına önce ad, sayfa sayısı ve `String` tipinde bir yazar yazdı, sonra şu soruyu sordu: Yazarı `String` olarak yazmak doğru mu? Cevabı şöyle verdi: Yazarın da kendi özellikleri olacağı için onun da ayrı bir sınıf olması gerekebilir. Bu, C'de iç içe yapı (nested struct) kurmaya benzer. Bu yüzden projeye bir `Yazar` sınıfı daha eklendi.

7.3 Sınıf Adında Tekil İsim

Hoca sınıfı önce `Kitaplar` adıyla oluşturmuştu, sonra Refactor ile `Kitap` yaptı. Gerekçesi şuydu: Kitap soyut bir kavramdır, somut hale gelmişine ise kitap nesnesi (object) deriz. Sınıf adı yazarken genellikle tekil isim kullanılır: `Yazar`, `Kitap` gibi.

> **Ek bilgi (derste anlatılmadı):** Hoca bu derste sınıfı "soyut", nesneyi "somut" diye andı. Buradaki "soyut" günlük anlamdadır. Büyük olasılıkla ileride Java'da `abstract` anahtar sözcüğüyle gelecek olan soyut sınıf ayrı bir kavramdır, karıştırılmamalıdır.

Sonuçta iki sınıf şu hale geldi. Alanların önüne henüz bir şey yazılmadı, erişim belirteçleri daha sonra anlatılacak:

```
package javaapplication3;

public class Yazar {
    String ad;
    int id;
}
```

```
package javaapplication3;

public class Kitap {
    String ad;
    int sayfaSayisi;
    Yazar yazar;   // yazar ayrı bir sınıf
}
```

Bu iki sınıf, Bölüm 4'teki `Deneme` ile birlikte javac ile derlenip çalıştırılarak denendi.

Bu sınıflar arasındaki ilişki düz metinle şöyle gösterilebilir:

```
Kitap                      Yazar
-----------------          ---------
ad : String                ad : String
sayfaSayisi : int          id : int
yazar : Yazar  ----------> (bir Yazar nesnesini gösterir)
```

Hocaya göre sonuçta her şey temel tiplerden oluşur.

> **Ders sonrası not:** Hoca nesnelerin "`string, integer, char` gibi" temel veri tiplerinden oluştuğunu söyledi; doğrusu Java'da `String` temel (primitif) tip değil, bir sınıftır. Java ile denendi: `String.class.isPrimitive()` sonucu `false`, `int.class.isPrimitive()` sonucu `true` çıktı.

7.4 Birden Fazla Yazar

Hoca, ileride koleksiyonları, listeleri ve dizileri görünce kitabın birden fazla yazarı olabileceğini de düşüneceğimizi söyledi. O zaman kitabın içinde bir yazarlar dizisi ya da listesi tutulur. Tek yazarlı kitap bir eleman, iki yazarlı kitap iki eleman saklar. Hoca bunu kısaca yazıp gösterdi, sonra kafa karıştırmamak için sildi ve tek yazarla devam etti.

8. Bellekte Nesne: Stack ve Heap

Sınıfı yazarak soyut bir şey tanımlamış olduk. Şu anda belli bir A, B, C ya da X yazarı yok, yalnızca "yazar" diye genel bir kavram var. Nesne oluşturmak, bunu somut hale getirmektir.

Somutlaştırınca bellekte yazar için bir alan açılır. Bu alanın altında sınıfın alanları kadar yer vardır. Hoca bunu dikey bir kutu olarak çizdi (kutuya örnek olarak bir de yaş yazdı, sonra bunun önemli olmadığını söyledi) ve içine değerlerin yazılacağını anlattı.

Bu alanın bellekte bir adresi vardır ve **heap** bölgesindedir. Nesnenin oluşturulma biçimi yüzünden heap'te tutulur. Bu adres de **stack** bölgesindeki bir değişkende saklanır, yani nesneye orada bir referans vardır. Hoca bunu "bir pointer gibi bir şey" diye anlattı. Yazar nesnesi için kullandığımız `y` değişkeni heap'teki yeri gösterir. Nesneyi oluşturan satır şudur:

```
Yazar y = new Yazar();
```

Sınıf eklemek ile nesne oluşturmak arasındaki fark böylece ortaya çıkar: Soyut tanım için projeye sınıf ekleriz, onu somut hale getirmek için de kod yazarız. Hoca `new` sözcüğünün ayrıntısına ileride geleceğini söyledi. Hocanın anlattığı bellek görünümü düz metinle şöyledir:

```
STACK                          HEAP
x : 5
y : (adres) -----------------> [Yazar nesnesi: ad, id]
```

9. Nesneyi Yazdırmak

Normalde bir değişkenin değerini yazdırmak için değişkeni yazarız. `int x = 5;` dedikten sonra `System.out.println(x);` doğrudan `5` yazar, çünkü `x` değer tipindedir ve hocanın anlatımına göre stack bölgesinde saklanır. `y` ise değer tipli değildir. Bu yüzden `System.out.println(y);` çalıştırıldığında ekranda "acayip bir şey" çıktı: Bölüm 4'teki çıktıda görülen `javaapplication3.Yazar@...` biçimindeki metin.

Hoca bu metnin nesnenin hangi sınıfa ait olduğunu ve referansını, yani bellek adresini yazmaya çalıştığını söyledi. Bunun neden böyle olduğunu ileride ayrıntılı anlatacağını, şimdilik kafa karıştırmaya gerek olmadığını ekledi.

> **Ders sonrası not:** Hoca çıktıdaki sayının nesnenin referansı ya da bellek adresi olduğunu söyledi; doğrusu bu, `toString()` metodunun yazdığı `hashCode()` değerinin onaltılık halidir. javac ile denendi: Bir `Yazar` nesnesi için `y.toString()`, `y.getClass().getName() + "@" + Integer.toHexString(y.hashCode())` ile aynı metni verdi. Python ile hesaplandı: Bir çalıştırmada çıkan `7344699f` değeri onluk `1933863327` eder ve bu, `hashCode()` değerine eşittir. Bu değer büyük olasılıkla nesnenin gerçek bellek adresi değildir.

> **Ders sonrası not:** Hoca değer tipli değişkenin stack'te saklandığını söyledi. Büyük olasılıkla bu yalnız `x` gibi yerel değişkenler için doğrudur. `sayfaSayisi` gibi bir nesnenin alanı olan `int` ise nesneyle birlikte heap'te durur.

Hocanın değindiği bir başka nokta da şuydu: `y.` yazınca NetBeans `ad` ve `id` alanlarını listeler. Bu alanların önüne ileride erişim belirteçleri gelecektir.

Soru: Yazar sınıfına nasıl özellikler eklemiştik? **Cevap:** Sınıfın içine `ad` ve `id` alanları yazılmıştı. Hoca yazara başka bir şey eklemediğini, yapısal programlamada olduğu gibi her şeyin sonunda temel veri tiplerinden oluştuğunu tekrarladı. Ama nesne üretiminde sonunda bir referans oluşur.

10. Nesne İçinde Nesne ve Nokta Operatörü

Kitap sınıfından bir nesne oluşturduğumuzda bellekte bir kitap kutusu oluşur. Hoca bu kutuyu çizerken içine ad, id ve yazar alanlarını koydu. Burada yazar artık soyut bir kavram değildir. Kitap somut olduğuna göre bu alanın da somut bir yazar nesnesiyle ilişkilendirilmesi gerekir. Hoca buradan şu sonucu çıkardı: Bir kitabın var olması için önce yazar nesnesinin oluşturulması gerekir. Daha önce oluşturulmuş yazar nesnesi heap'te durur, `y` onu gösterir, kitabın içindeki `yazar` alanı da aynı yeri gösterir. Kitap nesnesini de `k` değişkeni gösterir:

```
STACK                          HEAP
y : (adres) -----------------> [Yazar nesnesi: ad, id] <--------+
k : (adres) -----------------> [Kitap nesnesi: ad, ..., yazar] -+
```

`Deneme` içinde kitap nesnesi şöyle oluşturuldu:

```
Kitap k = new Kitap();
```

Bundan sonra `k.` yazılınca alanlar listelenir, `yazar` da bunlardan biridir. `k.yazar.` yazılınca da yazarın `ad` ve `id` alanları gelir. Yani `k.yazar.ad` ifadesi kitabın içindeki yazarın adına erişir. Hoca bunu `System.out.println` ile karşılaştırdı: Nasıl `System` altından `out`'a, oradan `println`'e gidiyorsak, burada da kitaptan yazara, yazardan yazarın bilgilerine gidiyoruz. Bu erişimin her zaman mümkün olmayacağını, buna daha sonra bakılacağını da ekledi.

> **Ders sonrası not:** Hoca bir kitabın var olması için önce yazar nesnesinin oluşturulması gerektiğini söyledi. Bu, kavramsal modeldir; Java bunu kendiliğinden zorunlu kılmaz. javac ile denendi: `new Kitap()` hatasız çalıştı ve `yazar` alanı `null` kaldı. Bu durumda `k.yazar.ad` çalışma anında `NullPointerException` verdi. Alana bir `Yazar` nesnesi atandıktan sonra erişim çalıştı.

11. Referans Mekanizması: Atama, Çöp Toplayıcı ve Kopyalama

> **Hocanın vurgusu:** Nesneye yönelik programlamayı kısaca özetleyin deseniz, o bir referanslar mekanizmasıdır. Her şey birbirine referans eder. Derin kopya çok azdır, çünkü maliyetlidir ve gerekirse yapılır.

Hoca bunu üç değişkenli bir senaryoyla ekranda adım adım çizdi. Senaryonun kodu şöyledir:

```
Kitap k = new Kitap();
Kitap k2 = k;
Kitap k3 = new Kitap();
k3 = k;
```

Satırların bellekte yaptığı şunlardır:

1. `Kitap k = new Kitap();` heap'te fiziksel olarak bir kitap nesnesi oluşturur. `k` onu gösterir.
2. `Kitap k2 = k;` satırında `k`'nın içindeki değerler fiziksel olarak başka bir yere gitmez. Yalnızca `k2` için yer açılır ve `k2` de aynı nesneyi gösterir.
3. `Kitap k3 = new Kitap();` fiziksel olarak yeni bir kitap için heap'te yer açar. `k3` bu ikinci nesneyi gösterir.
4. `k3 = k;` yapılınca `k3` artık ikinci nesneyi göstermez, `k`'nın gösterdiği yeri gösterir. İkinci nesneyle bağlantı kopar, nesne "başıboş" kalır.

Son durumun görünümü düz metinle şöyledir:

```
STACK                          HEAP
k  : (adres) ---+
k2 : (adres) ---+------------> [1. Kitap nesnesi]
k3 : (adres) ---+
                               [2. Kitap nesnesi]  (gösteren yok)
```

Hoca farkı şöyle özetledi: `new` ile yapılan işlem fiziksel olarak bir şey üretir, atama işlemi üretmez. Değişkenler belleğin stack bölgesindedir, `new` ile oluşan nesneler heap bölgesindedir.

Bu senaryo derste çizimle anlatıldı, kodla karşılaştırma yapılmadı. javac ile denendi: `k2 == k` sonucu `true`; atamadan önce `k3 == k` sonucu `false`, atamadan sonra `true` çıktı. `k.ad` değiştirildiğinde `k2.ad` da aynı değeri gösterdi.

11.1 Çöp Toplayıcı

Başıboş kalan alan ne olacak? C'de `malloc` ile aldığımız yeri `free` ile bırakıyorduk. Java'da bunun yerine **çöp toplayıcı (garbage collector)** vardır. Arada bir bakar, başıboş bir bellek alanı varsa temizler ve o bölgeyi serbest bırakır.

11.2 Derin ve Sığ Kopya

Hoca, atama işleminde kesinlikle **derin kopya (deep copy)** olmadığını vurguladı. Derin kopya maliyetlidir, nesneye yönelik programlamada daha çok **sığ kopya (shallow copy)** vardır. Derin kopya gerekirse buna klonlama denir ve nesneler arasında `clone` adında ayrı bir metot vardır. Hoca bu metodun, pointer'larda olduğu gibi bir nesnenin başladığı adresten öbür nesnenin başladığı adrese bütün değerleri bit bit ya da byte byte kopyaladığını söyledi. Ayrıntısına daha sonra geleceğini ekledi.

> **Ders sonrası not:** Hoca `clone` metodunun derin kopya yaptığını söyledi; doğrusu `Object.clone()` varsayılan olarak sığ kopya yapar. Nesnenin içindeki referans alanları aynı iç nesneleri göstermeye devam eder. javac ile denendi: `Cloneable` uygulayan ve `super.clone()` çağıran bir sınıfta `kopya == b` sonucu `false`, `kopya.yazar == b.yazar` sonucu `true` çıktı. `Cloneable` uygulamayan sınıfta aynı çağrı `CloneNotSupportedException` verdi. Derin kopya için iç nesnelerin de ayrıca kopyalanması gerekir.
>
> **Ders sonrası not:** Hoca sığ kopyayı `k2 = k` örneğiyle anlattı; doğrusu bu satır yalnız referans atamasıdır, ortada ikinci bir nesne yoktur (`k2 == k` sonucu `true`). Sığ kopyada ise yukarıdaki `clone` denemesinde görüldüğü gibi yeni bir nesne oluşur (`kopya == b` sonucu `false`), yalnız iç referanslar paylaşılır. İki sonuç da javac ile denendi.

12. Yapısal Programlamanın Java'daki Yeri

Hoca nesneye yönelik programlamayı bir küme olarak düşünmemizi istedi: OOP kümesi, yapısal programlamanın bütün özelliklerini içerir ve bunlara ek özellikler katar. Dolayısıyla geçen dönem C'den bilinen birçok şey burada da kullanılır:

- **Değişken tanımlamaları:** Normal değişkenler için aynıdır.
- **Diziler:** Tanımlama biçimi benzerdir, dizilerin ek özellikleri vardır. Sonraki derste kısaca anlatılacaktır.
- **`if`:** Hemen hemen yüzde 99 aynıdır.
- **`while` döngüsü:** Aynıdır.
- **`for` döngüsü:** Aynıdır. Tek fark, sayacın döngü içinde tanımlanabilmesidir: `for (int i = 0; ...; i++)`. Hoca C'de bunu kesinlikle yapmayın dediğini, burada ise tam tersine yapın diyeceğini söyledi.

  > **Ders sonrası not:** Hoca döngü içinde sayaç tanımlamayı C'de yapmayın demişti. Bu, C standardına bağlıdır. gcc ile denendi: `for (int i = 0; ...)` içeren kod `-std=c89 -pedantic-errors` ile hata verdi, `-std=c99 -pedantic-errors` ile derlendi. Java'da her zaman geçerlidir; javac ile denendi.

Bir kitabın çok yazarı, bir yazarın çok kitabı olabilir. Buna **çoka çok ilişki** denir. Hoca bu dersin öncelikli amacını şöyle anlattı: Nesneleri önce kavramsal ve teorik olarak oluşturmak, nesneler arasında ilişkiler kurmak ve bu ilişkilerin derecelerinden söz etmek. Bu konuların veri tabanında da yardımcı olacağını ekledi.

Hoca “Java öğrenmek” derken yabancı dil öğrenir gibi her şeye baştan, önce zamanları (tense) öğrenerek başlamayı düşünmememizi istedi. `if`, `for`, `while` yapısal programlamadan bilinir ve tekrar anlatılmayacaktır.

> **Hocanın vurgusu:** Önemli olan tasarımı doğru yapabilmek ve bunun mantığını doğru anlayabilmektir.

13. Debug: Programı Adım Adım Çalıştırmak

Debug için önce kodda duracağımız satıra bir **kesme noktası (breakpoint)** konur. Sonra **Debug** menüsünden **Debug Project** seçilir. Hocanın ekranında bu sırada hata çıktı, çünkü az önce örnek olarak yazılan `if`, `while` ve `for` yapılarının koşulları yazılmamıştı. Hoca bunun önemli olmadığını söyledi.

14. JFrame ve UML’in Ne Olduğu

14.1 JFrame ile Hazır Form

Hoca son olarak görsel programlamayı kısaca gösterdi. Uygulamaya **New** denip **JFrame** seçilince özel bir arayüz açılır. Burada bir **palet** vardır ve ileride basit görsel programlama için kullanılacaktır. Bu sınıfın yanındaki yeşil üçgen, sınıfta bir `main` metodu olduğunu gösterir. Kaynak koda bakıldığında gerçekten bir `main` görülür, ayrıca hocanın daha sonra anlatacağını söylediği, grafik arayüzle ilgili başka kodlar da vardır.

> **Hocanın vurgusu:** Hoca bu sürükle-bırak yaklaşımına karşı olduğunu, ilk zamanlarda bununla öğrenmenin iyi bir şey olmadığını söyledi.

Formun koduna bakıldığında sınıf tanımında daha önce yazdığımız sınıflardan farklı olarak `extends` ifadesi bulunur. Hoca buna **kalıtım** diyeceğimizi, konunun ileride anlatılacağını ve UML’de kendine özgü bir gösterimi olduğunu söyledi.

14.2 UML Nedir?

Soru: UML bununla mı ilgili? GUI ile alakalı gibi geldi. **Cevap:** Hayır, UML geniş bir kavramdır ve sonraki derste başlanacaktır. Düğme bir nesnedir, açılan pencere olan frame bir nesnedir; nasıl kitap ve yazar da birer nesneyse. UML’de kitabın nesne olduğu, altında hangi özelliklerin ve metotların bulunduğu belirtilir. Aynısı yazar için de yapılır. Ayrıca kitap ile yazar arasındaki ilişkisel model de ifade edilir: “Kitaplar yazarlar tarafından yazılır” gibi sözel bir ilişki UML ile gösterilir.

Hoca UML’i **bu işin modelleme dili**, yani **nesneye yönelik programlamanın akış diyagramı (flowchart)** olarak tanımladı.

15. IntelliJ’de Swing UI Designer ve PlantUML Denemesi

Hocanın IntelliJ’e bakacak zamanı olmamıştı, bu yüzden bir öğrenci ekranını paylaşarak IntelliJ tarafını gösterdi. Adımlar şöyleydi:

1. `src` klasörüne sağ tıklanıp **New** denildi. Burada JavaFX Application ve **Swing UI Designer** seçenekleri göründü.
2. Swing UI Designer altından **GUI Form** seçildi, isim verildi ve **OK** denildi. Hoca bunun JFrame karşılığı olduğunu söyledi.
3. Sağ taraftaki paletten bir **JButton** sürüklenip forma bırakıldı.
4. Kaynak koda bakıldı. Sınıf dosyasının içi neredeyse boştu. Hoca arayüz bilgisinin `.form` adlı ayrı bir yapılandırma dosyasında olması gerektiğini tahmin etti.
5. Form çalıştırılmak istendiğinde `main` metodu bulunmadığı görüldü. Hoca bunun için `JPanel` ile ilgili bir ekleme gerekeceğini, ileride bakacağını söyledi.

Bu denemede iki ortamın farkı da görüldü: NetBeans’teki JFrame formu `main` metoduyla geldi, IntelliJ’deki formda `main` kendiliğinden gelmedi.

Öğrenci ayrıca IntelliJ'in UML eklentisi olarak **PlantUML**'i önerdi. Çok indirilen bir eklenti olduğunu söyledi ve kurdu. Ancak kullanımını göstermek için açılan yeni proje denemesi derste tamamlanamadı. Hoca PlantUML'in kullanımına daha sonra bakılabileceğini söyledi.

16. Easy UML ile Sınıf Diyagramı

16.1 Eklentinin Elle Kurulumu

Hoca Easy UML'i eklenti listesinde bulamadı, bu yüzden indirdiği dosyadan elle kurdu. Eklenti bir zip dosyasıydı, içinde birden fazla `.nbm` dosyası vardı. Adımlar şöyleydi:

1. Zip dosyası masaüstünde bir klasöre açıldı.
2. NetBeans'te **Plugins** penceresine gidildi ve **Add Plugins** seçildi.
3. `.nbm` dosyalarının hepsi birlikte seçildi.
4. **Install**, **Next**, lisansların kabulü ve yeniden **Install** adımları izlendi.
5. Kurulumun başarıyla bittiği görüldü. UML menüsünün çıkması için NetBeans kapatılıp yeniden açıldı.

Hoca kurulumu önceden hazırlamak yerine derste birlikte yapmanın daha iyi olduğunu, böylece adımları hatırlayamayacak olanların da görmüş olacağını söyledi.

16.2 Koddan Diyagram Üretmek

Hoca önce projede hata veren satırları yorum satırına aldı. Sonra projeye sağ tıklayıp **Easy UML, Create Class Diagram** seçeneğini kullandı. Bu aşamada önce bir UML projesi gerektiği görüldü:

1. Yeni proje olarak **UML** altından **UML Diagram Project** seçilir, **Next** ve **Finish** denir.
2. Kaynak projeye dönülüp **Create Class Diagram** seçilir.
3. Diyagramın hangi UML projesine aktarılacağı sorulur, az önceki proje seçilir.

Sonuçta diyagramda `NewJFrame`, `Kitap` ve `Yazar` göründü, `Deneme` görünmedi. Hoca bunun, dosyanın hata vermesinden kaynaklanabileceğini tahmin etti.

Diyagramda sınıflar arasında çeşitli işaretler ve alanların yanında tilda (~) ve eksi (-) gibi semboller görülür. Hoca bunların hepsinin anlamına ileride bakılacağını söyledi.

> **Ek bilgi (derste anlatılmadı):** Büyük olasılıkla UML sınıf diyagramında + public, - private, # protected, ~ paket erişimini gösterir. Bizim `Yazar` ve `Kitap` alanlarının önüne hiçbir belirteç yazmadığımız için bu alanlar büyük olasılıkla diyagramda ~ ile göründü.

16.3 Diyagramdan Kod Üretmek

Yeni bir diyagram için **New, Other, UML, Class Diagram** seçilir, **Next** ve **Finish** denir. Boş diyagrama sınıf eklenir, sınıflara alanlar eklenir, başka bir sınıf daha eklenir ve sınıflar arasındaki ilişkiler (relation) yazılır. Ardından **UML, Generate Code** seçilerek kodun nereye aktarılacağı belirtilir. Hoca denemede boş bir sınıf (`UntitledClass`) üretildiğini gösterdi.

> **Hocanın vurgusu:** Önemli olan şu tasarımı yapabilmektir.

Soru: Hepimizin bir UML aracı indirmesi lazım değil mi? **Cevap:** Evet, bir şekilde UML çizme aracınız olmalıdır. NetBeans, Eclipse ya da IntelliJ olabilir, hangisi olduğu fark etmez.

Dersin Toparlaması

Bu derste geliştirme ortamı kuruldu ve bir Java projesinin yapısı görüldü: proje klasörü, `src`, paket klasörü ve `.java` dosyaları. Programın `main` metodu bulunan sürücü sınıftan başladığını, `println` ile `print` farkını ve `System.out.println` yazımının sınıf, statik nesne ve metot zinciri olduğunu öğrendik. Paketlerin projeyi mantıksal olarak böldüğünü ve aynı adlı sınıfları ayırt etmeyi sağladığını gördük.

Dersin merkezinde referans mantığı vardı. Sınıf soyut bir tanımdır, `new` ise heap'te somut bir nesne oluşturur. Değişken bu nesnenin referansını tutar, atama yeni nesne üretmez, hiçbir referansın göstermediği nesneyi çöp toplayıcı temizler. Yazarın ayrı bir sınıf olması ve `k.yazar.ad` erişimi, nesneler arasındaki ilişkilerin ilk örneğidir. Sonraki derste UML'e başlanacak, dizilere kısaca değinilecek, erişim belirteçleri, kalıtım ve nesneler arası ilişki türleri dönem içinde ayrıntılı işlenecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.